

UI/UX Deliverables and the Frontend

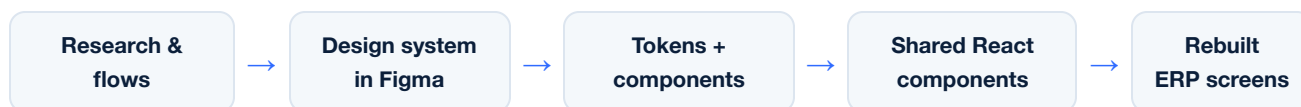
Understanding the UI/UX specialist role and how its output feeds this ERP's frontend

Prepared September 20, 2026

WHY THIS DOCUMENT

You're new to the UI/UX side of this project. This is a plain-language map of what a UI/UX specialist actually produces, and exactly where each thing lands in `client-app/` — so the work doesn't feel abstract when it starts arriving.

THE PIPELINE, END TO END



The UI/UX specialist works furthest from the code (left side); the frontend developer works closest to it (right side). The middle two steps — design tokens and a component library — are the handoff point where design decisions turn into things a developer can literally import.

THE MAIN ARTIFACTS, ONE BY ONE

1. Research & information architecture (IA) — user flows, navigation structure, sitemaps. Answers "what screens exist and how does a user move between them," before any visual design starts.

Lands in the repo as: route structure and menu definitions (e.g. `SalesRequestMenu` and its sibling `*Menu.tsx` files) — the Menu-Exit pattern in this codebase exists precisely because IA and routing don't always line up 1:1 here.

2. Wireframes — low-fidelity, greyscale layout blocks. Settle where things go on a screen before anyone argues about color or font.

Lands in the repo as: the rough layout/grid structure of a feature screen, before it's styled with Kendo/MUI components.

3. Design system (Figma file) — the reusable rulebook: color and type styles, a spacing/radius/shadow scale, and primitive components (Button, Card, Badge, etc.) with their variants. This is the category the `iCoreSprint-design-system.pdf` and the Figma rebuild task belong to.

Lands in the repo as: design tokens (CSS variables / a shared theme object) applied on top of the two UI kits we're keeping (Kendo + MUI), and a small shared component layer (Menu-Exit, Actions Menu, Status Ribbon) built once in Storybook instead of copy-pasted per screen.

4. High-fidelity mockups — pixel-accurate, fully styled screens for specific ERP features (e.g. what the redesigned Sales Order form should actually look like), using the design system's styles and components.

Lands in the repo as: the visual target a developer implements a given feature screen against — the "before" is the current live screen, the "after" is this mockup.

5. Redlines / Dev Mode specs — exact measurements pulled from Figma (padding in px, font sizes, hex codes, corner radii) so a developer isn't eyeballing a screenshot.

Lands in the repo as: the literal numbers used in component styling (MUI `sx` props, Kendo theme variables) — this is why the design-system PDF calls out which values are "exact" (colors) versus "estimated" (type/spacing): estimated values are not safe to treat as redlines yet.

6. Accessibility annotations — required ARIA roles, keyboard focus order, color-contrast notes.

Lands in the repo as: `aria-*` attributes and keyboard-nav handling on components — a known gap in this codebase (roughly 93 of 922 component files have any `aria-*` coverage as of the last audit).

7. Clickable prototypes — wireframes or mockups linked together so a flow can be tested before any code is written, catching UX problems while they're still cheap to fix.

Lands in the repo as: nothing directly — its value is upstream, reducing rework once a developer starts building the real screen.

WHERE THIS FITS IN OUR MODERNIZATION PLAN

This ERP's frontend modernization is organized into five tracks. UI/UX deliverables map almost entirely onto **Track D — design tokens and a shared component layer**, with a smaller overlap on Track A:

- **Track A (consolidate, don't replace)** — UI/UX work styles the two UI kits we're keeping (Kendo for grids/forms, MUI for layout/dialogs); it does not introduce a third kit. A design system that assumes a totally different component library would conflict with this track.
- **Track D (design tokens + shared components)** — this is the direct destination for design-system output: a spacing scale, one date-picker pattern, one confirmation-dialog pattern, and the Menu-Exit / Actions Menu / Status Ribbon components, promoted from "documented convention in CLAUDE.md" to actual shared, Storybook-documented components.

Concrete status right now: `iCoreSprint-design-system.pdf` is a stand-in brand book (reverse-engineered from screenshots, not yet this ERP's own confirmed brand). The companion task assigns a new team member to turn it into a real Figma file. Once that Figma file exists, the next step is extracting its tokens into this codebase's theme layer and building the Track D shared components against it — not redesigning individual ERP screens yet.

ARTIFACT-TO-CODE QUICK REFERENCE

ARTIFACT	WHAT IT IS	WHAT IT BECOMES IN THE FRONTEND	COMMON TOOLS BESIDES FIGMA
User flows / IA	Navigation & screen map	Routes, menu definitions	FigJam, Miro, Whimsical, Lucidchart
Wireframe	Low-fi layout	Screen layout/grid, before styling	Balsamiq, Sketch, Adobe XD
Design system (Figma)	Colors, type, spacing, primitives	Design tokens (theme.ts / CSS vars), Kendo+MUI theme overrides	Tokens Studio (Figma plugin) or Style Dictionary to export tokens as JSON/CSS/TS; zeroheight to publish the rulebook as docs
Component library (Figma)	Reusable UI pieces + variants	Shared React components in Storybook (Menu-Exit, Actions Menu, Ribbon)	Storybook (where the built components actually live/get documented on the dev side)
High-fidelity mockup	Pixel-accurate screen	Implementation target for a specific feature screen	Sketch, Adobe XD (Figma alternatives, same role)
Redlines / Dev Mode	Exact measurements	Literal spacing/size/color values in component code	Figma Dev Mode (built in), Zeplin, Avocode
Accessibility annotations	ARIA, focus order, contrast	<code>aria-*</code> attributes, keyboard handling	Stark (Figma plugin, contrast/annotations), axe DevTools, WAVE, Lighthouse (dev-side auditing)
Clickable prototype	Testable flow, no code	Nothing directly — prevents rework upstream	InVision, ProtoPie, Framer

Figma covers most of the design-side column on its own (wireframes, high-fidelity screens, prototyping, Dev Mode) — the tools above are either the traditional alternative to a given Figma feature, or a separate tool used alongside Figma for a specific job (token export, documentation, or accessibility testing) that Figma itself doesn't do.